- <technique>
  - <AI(Artificial Intelligence>
    인간지 지능을 모방하는 소프트웨어를 말하는 추상적 개념
    - <machine learning>
      데이터분석기 예측을 위해 학습하는 AI 방법론
      [지도 학습]
      (오답과 정답이 주어지는 데이터로, 입력에서 정답을 예측하는 방법 학습
      [비지도 학습]
      (정답 없이, 데이터 자체 구조나 분포에서 패턴 학습
      [강화 학습]
      (환경에서 행동하며, 보상을 최대화하는 방법 학습
      - <classical ML>
        사람이 데이터의 특징을 정의하고, 특징을 근거로 예측을 학습하는 ML
      - <deep learning>
        데이터의 특징 추출까지 스스로 수행하는 ML
        - <transformer>
          전체 데이터 간의 관계 파악을 학습하는 아키텍처
          - <Language Model>
            텍스트 데이터로 언어 패턴을 학습한 모델
            [reasoning model]
            생각 추론을 학습하여 출력하고 이전 추론이 입력에 학습해야 모델
            - <Relational-Augmented Generation>
              관련 지식베이 관련 정보를 검색하여, LLM 입력에 포함시키는 기법
              [vector search]
              텍스트 의미 유사도 기반 검색
              [keyword search]
              단어 일치 및 중요도 기반 검색
              [graph search]
              graph DB 검색
              [SQL search]
              RDBMS 기반 SQL 생성 및 쿼리 검색
              [web search]
              - <agent architecture pattern>
                [ReAct (Reasoning+Acting)]
                thought-action-observation 루프 반복
                [plan-and-execute]
                전체 계획 수립 → 단계별 실행 → 재검토/수정계획
                [Swarm]
                개별 수행하도록 전문 에이전트 생성 → 결과 종합
                [reflection]
                실행 → 오류/성능 평가 → 문제 원인 분석 → 재시도 → 학습
          - <Open AI>
            지식베이스 없이 LLM 자체 학습 모델
          - <Llama>
            오픈소스 기반 LLM 자체 학습 모델
          - <Gemini>
            구글 DeepMind 개발 멀티모달 모델
          - <Claude>
            Anthropic 개발 안전성 중심 대화형 모델
          - <Grok>
            xAI 개발 실시간 웹 접근 특화 모델
          - <Mistral>
            유럽 오픈소스 기반 효율적 모델
          - <Qwen>
            Alibaba 개발 다국어 특화 모델
          - <DeepSeek>
            중국 오픈소스 기반 코딩 특화 모델
        - <AI agent>
          자율성 있는 LLM 기반 시스템, 목표 달성을 위해 도구 사용 및 의사결정 수행
        - <Llama Model>
          메타 개발 오픈소스 기반 범용 모델
        - <Audio Model>
          음성 데이터 기반 음성 인식 및 생성 모델
        - <Image Generation Model>
          이미지 데이터로 시각 정보 생성을 학습한 모델
        - <video Generation Model>
          영상 데이터로 시간+시간 정보 생성을 학습한 모델
    - <knowledge-based>
      사람의 해법을 수동으로 정의하는 AI 방법론
  - <Database Management System>
    데이터베이스를 관리하는 소프트웨어
    [Database]
    과정 시스템의 중속성, 중복성 문제를 해결하기 위해 고안된 데이터 저장 방식,
    [Relational Database]
    (행과 열 형태로 데이터를 저장하는 관계형 데이터베이스 방식)
    - <Relational DatBase>
      데이터를 정형화하여 테이블 구조로 구축하고 테이블 간 관계를 정의하여 사용한다.
      - <DDL>
        (Structured Query Language)
        relational database language
        [transaction]
        DB process unit
        -Atomicity: commit || rollback
        -Consistency: constraint
        -Isolation: race condition prevent
        -Durability: logging
        [trigger]
        데이터, 상태가 변경되었을 때 자동적 동작하는 트리거이다
        - <DDL>
          (Data Define Language)
          - <DDL keywords>
            [CREATE] 생성 구조 생성
            [CREATE field Type fieldname ...]
            [ALTER] 필드 구조 변경
            [ALTER field Type fieldname ADD | ALTER | DROP COLUMN]
            [DROP] 필드 구조 삭제
            [DROP field Type fieldname CASCADE | RESTRICT]
            - <Database structure>
              SCHEMA, DOMAIN, VIEW, INDEX, TABLE
              [SCHEMA]
              전반적인 구조(인덱스, 뷰, 테이블, 도메인 그룹화) 정의
              [DOMAIN]=AS type CHECK(제약조건)
              제약조건을 사용하여 사용자 정의 속성 타입 생성
              [INDEX]=ON tablename(field)
              binary search 등의 방법을 위해 적절한 방법을 통한 정렬해두는 별도 자료 구조
              [VIEW]=AS SELECT – FROM –[-
              -SELECT 문으로 조회하여 구축한 가상 테이블
              [TABLE]=Expor attributeName, ..., CHECK(제약조건), ...]
              mapping(attribute, domain) 제약조건 정의
              -key: primary key(기본키), foreign key(외래키)
              -key: fuplicate 구분가능, 고유성 값들과 이후비교 속성값
              - Binary Search
              이진 탐색 알고리즘 중 하나로, 정렬된 배열에서 특정 값을 찾기 위해 중간 값을 기준으로 절반씩 범위를 좁혀가는 방식이다. 시간 복잡도는 O(log n)이며, 선형 탐색보다 훨씬 빠르지만 데이터가 반드시 정렬되어 있어야 한다.
            - <DCL keywords>
              [GRANT] 권한 부여
              [REVOKE] 권한 회수
              [ROLLBACK] 명령어 수행 이전으로 복귀
              [SAVEPOINT] ROLLBACK 위치 지정
              [GRANT] 명령어 관련 옵션
              [GRANT 권한명 ON 테이블 TO 대상 (WITH GRANT OPTION)]
              [REVOKE] 명령어 관련 옵션
              [REVOKE [GRANT OPTION FOR] 권한명 ON 테이블 FROM 대상
        - <DML>
          (Data Manipulation Languages)
          - <DML keywords>
            [SELECT] 테이블 내 레코드 검색(추출)
            [INSERT] 테이블 내 레코드 삽입
            [INSERT INTO 테이블(필드명, ...) VALUES(데이터값2, ...)]
            [UPDATE] 테이블 내 특정 레코드 변경
            [UPDATE 테이블 SET 속성명=변경값 WHERE 조건식]
            [DELETE] 테이블 내 레코드 삭제
            [DELETE FROM 테이블 WHERE 조건식]
            - <SELECT>
            SELECT [DISTINCT] [table_name.]{ * | attrname1, [] group function,...
            FROM table [AS rename],
            *조건
            [WHERE [NOT] condition]
            [WHERE condition AND | OR condition]
            [WHERE attribute IS [NOT] NULL]
            [WHERE attribute [NOT] IN (subquery)]
            [WHERE attribute BETWEEN condition AND condition]
            [WHERE condition IS [NOT] NULL]
            *순서 정렬
            [ORDER BY attribute ASC(오름차순) | DESC(내림차순), ...]
            *그룹핑 집합 정렬
            [GROUP BY attribute, ...HAVING condition]
            *다중 결합(내부 외부)
            [UNION(합집합) | INTERSECT(교집합) | MINUS(차집합)]
            INTERSECT(교집합) | EXCEPT(차집합)]
            SELECT attribute, ..FROM table_[
            *조인 결합(내 외부)
            [INTER JOIN(조인)]
            [CROSS JOIN(교차조인)]
            LEFT OUTER JOIN(왼쪽조인)]
            RIGHT OUTER JOIN(오른쪽조인)]
            FULL OUTER JOIN(완전조인)]
            CROSS JOIN(교차조인)
            table ON condition...]
            - <aggregate function>
              [COUNT(attribute)] 개수
              [SUM(attribute)] 합
              [AVG(attribute)] 평균
              [MAX(attribute)] 최대값
              [MIN(attribute)] 최대값
              [STDDEV(attribute)] 표준편차
              [VARIANCE(attribute)] 분산
              [ROLLUP(attribute, ...)] 소계+총계
              [CUBE(attribute, ...)] 총계+소계
            - <like wildcard character>
              [%] 모든 문자
              [_] 단일 문자
        - <DCL>
          (Data Control Language)
          - <DCL keywords>
            [COMMIT] 명령어 수행 결과 저장
            [ROLLBACK] 명령어 수행 이전으로 복귀
            [SAVEPOINT] ROLLBACK 위치 지정
            [GRANT] 명령어 관련 옵션
            [GRANT 권한명 ON 테이블 TO 대상 (WITH GRANT OPTION)]
            [REVOKE] 명령어 관련 옵션
            [REVOKE [GRANT OPTION FOR] 권한명 ON 테이블 FROM 대상
  - <Relational Database>
    데이터를 정형화하여 테이블 구조로 구축하고 테이블 간 관계를 정의하여 사용한다.
    [transaction]
    DB process unit
    -Atomicity: commit || rollback
    -Consistency: constraint
    -Isolation: race condition prevent
    -Durability: logging
    [trigger]
    데이터, 상태가 변경되었을 때 자동적 동작하는 트리거이다
- <design>
  - <planning>
    [Product Requirements Document]
    - 요구 조사와 할 기능 정의
    [Work Breakdown Structure]
    - 8~80시간 단위로 작업 분할
  - <UI/UX design>
    - 구상 요소, 전략 초안 정리
    [prototype]
    - wire frame -> mockup -> prototype
  - <software design>
    [flow chart]
    - 소프트웨어가 해결한 목표 초안 정리
    [Entity-Relationship Diagram]
  - <hardware design>
    [system architecture]
    - 하드웨어, 네트워크 설정 전략 구조 정리
- <software>
  - <API>
    (Application Programming Interface)
    software-software interface
  - <Software Development Life Cycle>
    - <waterfall>
      - <requirement>
        - finally integrated component
        [code] -opcode(logic) + operand(value, address)
        [data] -value + type
        [file] -code in drive
        [program] -runnable file
        [process] -program in memory
        [module] -functional unit code
        [component] -service unit code
        [package] -group of unit codes
        [library] -reusable detail package
        [framework] -reusable structure package
        - <implementation>
          - <opcode>
            (사람이 개선할 코드)
            - <control statement>
              [conditional statement]
              -if(else)
              -switch(case)
              [iteration statement]
              -for
              -while
              -do-while
              [branch statement]
              -return: 함수를 종료하고 반환값을 대입한다.
              -break: 해당문을 빠져나온다.
              -continue: 현재 반복만 종료한다.
              - <Exception>
              [try] 실행할 코드
              [catch] try에서 오류 발생 시 실행할 코드
              - <try-catch>
              [try] 실행할 코드
              [catch] try에서 오류 발생 시 실행할 코드
              - <operator>
              [increment & decrement]
              ++x(1 증가) --y(1 감소)
              [arithmetic]
              *곱셈(*),나눗셈(/),모듈러스(%)
              +,-(더하기, 빼기)
              [relational]
              <(작다), >(크다), <=(작거나 같다.)
              >=(같다.) !=(다르다.) ==(같다.)
              [logical]
              !(NOT) &&(AND) ||(OR)
              [bit]
              &(and) ^(xor) |(or) ~(not)
              <<(left shift) >>(right shift)
              (floating point) 비교하지 않으시오.)
              [ternary]
              ? (true) : (false)
              [assignment]
              =(주어 줘)...
              - <algorithm>
              (문제의 본질)
              [logic]
              ,AND 조합
              - <search algorithm>
              [for search]
              전체 값에 접근하여 원하는 값을 찾는 방법
              [binary search]
              정렬된 데이터에서 중간값과 비교하여 절반씩 범위를 좁혀가는 효율적인 방법
              [hash search]
              해시 테이블을 이용해 직접 위치를 계산하여 빠르게 값을 찾는 방법
              - <graph algorithm>
              [Depth First Search]
              -깊이 우선
              -스택 사용
              -루트 순회
              [Breadth First Search]
              -폭우선 탐색에서 모든 정점을 레벨 단위로 탐색
              [Minimum Spanning Tree]
              [shortest path]
              - <string search algorithm>
              [brute force]
              -단순하고 한 문자씩 전체 탐색
              [Boyer Moore]
              -문자열 내에서 패턴을 찾는 효율적인 알고리즘
              -뒤에서부터 패턴과 비교하며 불일치 문자를 만나면 패턴을 최대한 건너뛰는 방식으로 동작하며, 평균적으로 매우 빠른 검색 속도를 보여준다.
              - <sorting algorithm>
              [insertion sort] [O(N^2)]
              -삽입 정렬 알고리즘
              정렬되지 않은 요소를 하나씩 꺼내 이미 정렬된 부분의 알맞은 위치에 삽입하는 방식
              [selection sort] [O(N^2)]
              -선택정렬 최소값을 찾아 맨 앞에 교체 하는 과정
              [bubble sort] [O(N^2)]
              -인접한 한쌍 씩 비교해 교체 정렬을 해주는 단순한 방식 값들이 거의 이미 정렬되어 있을 때 효율적이다.
              [quick sort] [O(NlogN)~O(N^2)]
              -피벗 값 기준 정렬하는 구조 값이 모두 정렬되는 방식
              [merge sort] [O(NlogN)]
              -안정 요소가 발생하지 않는 안정적인 정렬 방법
              -분할정복 방식으로 배열을 절반씩 나눈 뒤 병합하는 방식
              - <recursive algorithm>
              -자기 자신을 반복하여 문제 해결 방법이다.
              -비슷 하거나 동일한 문제를 작은 단위로 나누어 해결하는 방식이다.
              -스택 오버플로우 문제가 발생할 수 있으며, 메모리 효율성 및 성능 최적화가 중요하다.
            - <loop>
            [while]
            [for] 실행할 코드
            [do-while]
            [for(var in)] ...
            - <operand>
            (사용되는 데이터)
            - <variable>
            [variable] 데이터를 담을 변수
            [constant] 변하지 않는 값
            [literal] 코드에 직접 입력된 값
            [expression]개별 변수 및 연산자를 조합하는 방식
            - <data>
            [primitive] (deep copy)
            -int(정수), float(실수)
            -boolean(T/F), char(문자)
            [reference] (shallow copy)
            -array, object, function, type, pointer
            -address: object address
            -structure: structure address
            -class: logic+structure address
            - <structure>
            - <array>
            - continuous [index] of same primitive type
            [index] -> data + start_address
            - <Global Var>
            [static] -> data + start_address
            - <embedded structure>
            [list] 순서대로 데이터가 중복해 저장되는 배열
            [set] 순서없이 데이터가 중복해 저장되지 않는 배열
            [hashset] [key, value]
            -key: 해시 함수를 통해 변환된 인덱스 값
            -hash: 해시 함수가 생성한 고유 정수 값
            -a: 실제 배열 위치 인덱스 값을 가리키는 숫자 (0부터 시작)
            [stack] LIFO (Last-In First-Out)
            [queue] FIFO (First-In First-Out)
            [graph] Vertex + Edge
            -vertex: node + data + parent_address + children_address
            -binary tree: limit child_addresses to a maximum of 2
            - <memory management>
            - <Typing(Type checking)>
            [static typing] compile-time typing
            [dynamic typing] run-time typing
            - <memory allocation>
            [static allocation] compile-time allocation
            [dynamic allocation] run-time allocation
            - <lifetime>
            [local lifetime] lifetime in call stack
            [global lifetime] lifetime in run-time
            - <scope>
            [static scope (lexical)]
            -Dotch scoped: 변수는 선언된 블록을 그 자식 블록에서만 유효하다.
            -Dunction scope: 변수는 선언된 함수와 그 자식 함수에서만 유효하다.
            [dynamic scope]
            호출되어 스택에 남아 있는 동안 유효하다.
            - <Runtime>
            - 자신이만 기동하여 일을 반복하는 과정
            [Heap]
            -전역(자기 자신이) 고유하게 관리되는 영역 코드에 없다.
            -스택 코드와 힙간의 주고받기 과정이 있다.
            -개별 요구하면 코드들을 힙에 저장하고 일부를 스택 방식 영역 생성
            - <Stack>
            - 스스 코드들 이 자신 컴파일이 필요한 상황 코드로 변환하는 과정
            - <Heap>
            - 자신이만 기동하여 일을 반복하는 과정
            [Stack]
            -전역(자기 자신이) 고유하게 관리되는 영역 코드에 없다.
            -스택 코드와 힙간의 주고받기 과정이 있다.
            -개별 요구하면 코드들을 힙에 저장하고 일부를 스택 방식 영역 생성
            - <Stack_Heap>
            - 스스 코드들 이 자신 컴파일이 필요한 상황 코드로 변환하는 과정
            - <test>
            [unit test]
            test in module
            [integration test]
            test between modules
            [End to2 End test]
            [acceptance test]
            -alpha test
            -beta test
            - <white box test>
            [statement coverage]
            -모든 구문 수행
            [branch coverage]
            -모든 분기 경로 수행
            [condition coverage]
            -모든 조건식 참/거짓 수행
            - <black box test>
            [equivalence partitioning]
            -유효하거나 잘못 테이블을 효율적 테스트하기 위한 기법을 입력하여 테스트
            [boundary value analysis]
            -유효한 범위 내 경계값을 입력하여 테스트
            [cause effect graph]
            -다양 입력에 대한 다양 출력의 관계를 그래프로 표시
            - <deployment>
            -최종 서비스에 대한 인터페이스를 제공하는 과정
            [runtime environment]
            -정기적 리소를 교체할 수 있는 환경
            - <packaging>
            -빌드 결 포함하여, 관리있는 환경과의 테스트 배포 단위로 묶는 과정
            - <maintenance>
            [monitoring]
            -로그 등을 분석 장애 기록
            [Software Configuration Management]
            -실제명 관리 버전별 관리하고 변경점 수 있는 도구
            [Continuous Integration/Continuous Deployment]
            -빌드-테스트-배포 자동화 개념
            [release notes]
            -릴리즈마다 신뢰성 이슈(planned | added | changed | fixed)
    - <processing paradigm>
      -어떻게 사용할 것인가
      - <declarative programming languages>
      [what to do]
      (how' is implemented by system.)
      - <imperative programming language>
      "what" + "how"
      - <Object-Oriented Programming>
      -OO: 객체
      -사물처럼 생각하기에 문제 해결
      - <core functions>
      -기능 별 정는 단위를 갖춘 재사용성 높은 함수다. 사람별 함수를 통해 동시 실행되는 경우 데이터 경쟁이 발생하지 않는다. 입, 출 시간이 고려를 통해 최대한이라도 병렬처리는 방지 있다.
      (SIMD)는 같은 명령을 여러 데이터에 동시에 적용하는 방식이다. 벡터 연산, 이미지 처리, 과학 계산에 주로 사용되며 CPU 수준에서 성능을 극대화한다.
      [immutability]
      한 번 초기화된 데이터는 다시 변경할 수 없다. 데이터 경쟁(Race Condition)을 원천 차단하여 멀티스레드 환경에서 안전하게 동작한다. 함수형 프로그래밍의 핵심 원칙이다.
      [pure function]
      -동일한 입력에 대해 항상 동일한 결과를 반환하는 함수다. 부수 효과(Side Effect)가 없어 테스트하기 매우 쉽고 병렬 처리에 유리하다.
      - <Functional Programing>
      -FP: 함수
      -수학적 관계 해소
      - <Procedural Programing>
      -PP: 절차
      -시퀀스 흐름 해소
      - <sequential programing>
      - <Function>
      -선택 흐름 구조(조건식 참/거짓 흐름 분기)
      -loop: 반복 흐름 구조(조건식 참/거짓 흐름 반복)
      -arguments: 파라미터 함수에 전달되는 데이터
      -return value: 파라미터 보내는 내부값
      - <structure>
      변수에 대한 유효범위
      - <try-catch>
      [try] 실행할 코드
      [catch] try에서 오류 발생 시 실행할 코드
      - <control statements>
      [conditional statement]
      -if(else)
      -switch(case)
      [iteration statement]
      -for
      -while
      -do-while
      [branch statement]
      -return: 함수를 종료하고 반환값을 대입한다.
      -break: 해당문을 빠져나온다.
      -continue: 현재 반복만 종료한다.
      - <Exception>
      [try] 실행할 코드
      [catch] try에서 오류 발생 시 실행할 코드
      - <operator>
      [increment & decrement]
      ++x(1 증가) --y(1 감소)
      [arithmetic]
      *곱셈(*),나눗셈(/),모듈러스(%)
      +,-(더하기, 빼기)
      [relational]
      <(작다), >(크다), <=(작거나 같다.)
      >=(같다.) !=(다르다.) ==(같다.)
      [logical]
      !(NOT) &&(AND) ||(OR)
      [bit]
      &(and) ^(xor) |(or) ~(not)
      <<(left shift) >>(right shift)
      (floating point) 비교하지 않으시오.)
      [ternary]
      ? (true) : (false)
      [assignment]
      =(주어 줘)...
      - <algorithm>
      (문제의 본질)
      [logic]
      ,AND 조합
      - <search algorithm>
      [for search]
      전체 값에 접근하여 원하는 값을 찾는 방법
      [binary search]
      정렬된 데이터에서 중간값과 비교하여 절반씩 범위를 좁혀가는 효율적인 방법
      [hash search]
      해시 테이블을 이용해 직접 위치를 계산하여 빠르게 값을 찾는 방법
      - <graph algorithm>
      [Depth First Search]
      -깊이 우선
      -스택 사용
      -루트 순회
      [Breadth First Search]
      -폭우선 탐색에서 모든 정점을 레벨 단위로 탐색
      [Minimum Spanning Tree]
      [shortest path]
      - <string search algorithm>
      [brute force]
      -단순하고 한 문자씩 전체 탐색
      [Boyer Moore]
      -문자열 내에서 패턴을 찾는 효율적인 알고리즘
      -뒤에서부터 패턴과 비교하며 불일치 문자를 만나면 패턴을 최대한 건너뛰는 방식으로 동작하며, 평균적으로 매우 빠른 검색 속도를 보여준다.
      - <sorting algorithm>
      [insertion sort] [O(N^2)]
      -삽입 정렬 알고리즘
      정렬되지 않은 요소를 하나씩 꺼내 이미 정렬된 부분의 알맞은 위치에 삽입하는 방식
      [selection sort] [O(N^2)]
      -선택정렬 최소값을 찾아 맨 앞에 교체 하는 과정
      [bubble sort] [O(N^2)]
      -인접한 한쌍 씩 비교해 교체 정렬을 해주는 단순한 방식
      [quick sort] [O(NlogN)~O(N^2)]
      -피벗 값 기준 정렬하는 구조 값이 모두 정렬되는 방식
      [merge sort] [O(NlogN)]
      -안정 요소가 발생하지 않는 안정적인 정렬 방법
      -분할정복 방식으로 배열을절반씩 나눈 뒤 병합하는 방식
      - <recursive algorithm>
      -자기 자신을 반복하여 문제 해결 방법이다.
      -비슷 하거나 동일한 문제를 작은 단위로 나누어 해결하는 방식이다.
      -스택 오버플로우 문제가 발생할 수 있으며, 메모리 효율성 및 성능 최적화가 중요하다.
    - <processing systems>
      - <process copy>
      [eager copy] copy to access point
      [write copy-on-write] copy at modified point in time while referencing
      - <task execution model>
      [synchronous]
      -순차적 실행 방식 (한 작업이 끝나야 다음 작업을 시작함)
      [asynchronous]
      -비동기 실행 방식 (작업 완료 여부와 관계없이 다음 작업을 바로 시작함)
      [non-blocking]
      -호출자가 작업을 즉시 반환받아 다른 일을 할 수 있음
      [blocking]
      -호출자가 작업을 끝마칠 때까지 대기함
      - <non-blocking asynchronous model>
      [task queue]
      -작업자의 요청 목록
      [event loop]
      -이벤트 발생 감지
      -호출자가 작업을 즉시 반환받아 다른 일을 할 수 있음
      - <task_parallelism>
      [task parallelism] 여러 프로세스를 동시에 실행하는 것을 말하다. 서로별 할 일을 분리하여 각 프로세스에 할당하는 것이다. 일 시간이 고려를 통해 최대한이라도 병렬처리는 방지 있다.
      (SIMD)는 같은 명령을 여러 데이터에 동시에 적용하는 방식이다. 벡터 연산, 이미지 처리, 과학 계산에 주로 사용되며 CPU 수준에서 성능을 극대화한다.
      [synchronization]
      -동기화 메커니즘은 여러 프로세스나 스레드가 공유 자원에 접근할 때 데이터 경쟁(Race Condition)을 방지하기 위해 사용되는 핵심 기법이다. 뮤텍스, 세마포어, 모니터 등이 대표적인 동기화 수단이며, 교착 상태(Deadlock)와 기아(Starvation) 문제를 고려해야 한다.
      (1) 자원에 접근 시도 → 사용 → 해제 순서를 반드시 지켜야 함 (2) 잠금 획득 순서를 일관되게 유지하여 교착 상태 방지 (3) 가능한 한 짧은 시간만 잠금을 유지하여 성능 저하 최소화
      [multi-core] 멀티코어는 프로세서 내부에 여러 개의 독립적인 코어를 탑재하는 하드웨어 아키텍처를 말하며, 각 코어는 자체 연산 장치와 캐시를 가지고 있어 동시에 여러 작업을 처리할 수 있다. 이는 멀티스레딩 및 병렬 프로그래밍의 기반이 되는 하드웨어 구조다.
      - <processing Method>
      [time-during -(compression algorithm)]
      -하나 이상 내용 중복되어 정보량 변화가 없는 경우에 오답을 방지하기 위한 방법
      [Single-stream lossh processing compression (compressor algorithm)]
      -사전에 학습된 문법에 대해 압축하는 방법 있다가 있는 방식
      - <fork>
      -사용할 방식이 (each 개별 프로세스 생성하는 프로세스는 정보 자료구조)
      [Process Table]
      [Program Counter]
      [Memory Allocation]
      [Process ID]
      [Priority] scheduling
      [Parent PID]
      [Resource Usage Information] (file, I/O, ...
      - <Process_state>
      [new state] 프로그램이 메모리에 올라오고 PCB를 할당받은 상태
      [ready state] CPU 스케줄러가 지정한 다음에 의해 실행 대기 중인 상태
      [running state] 프로세스가 CPU를 점유하면 실행되는 상태
      [waiting state] 실행 상태가 프로세스가 입출력 도착에 의한 비차단 상태
      [terminated] 프로세스가 종료되고 할당된 메모리해제를 반납하는 상태
      - <Interrupt>
      [Synchronous Interrupt]
      CPU(Instruction cycle)중 하나 가로채는 신호
      (문제 고쳤 시 처럼 모방 공짜 (exception)) 시스템 호출 ...)
      [asynchronous Interrupt]
      -CPU와의 연결되는 외부 장치 중일
      (타이밍 기다리는 시간 동안 CPU는 다른 작업을 수행하고 CPU는 주기적으로 이벤 확인하여 다른 비스 버스에 요청 중인지 (이) 한 경우 시 시지하여 인터 이...)
      - <memory_management>
      [memory leak]
      -메모리 교체되 CPU 없는 컴퓨팅 장치에서 CPU가 다른 일을 못하는 현상
      - <memory_trade-off>
      [virtual memory] use drive as memory (swap)
      - <Paging>
      -페이지 교체의 CPU 컴퓨팅 장치에서 CPU가 다른 일을 못하는 현상
      - <memory partition>
      [segment] 논리 분할 방식
      -고정된 단위(segment) 형태 메모리를 분할하는 방식
      [segmentation] 가변 분할 방식
      -프로세스 크기에 맞게 메모리를 할당하는 방식
      - <process memory layout>
      [kernel area] OS code(swapping table, ...)
      [code area] -opcode + operand address
      [data area] -static allocation, global lifetime
      [heap area] -dynamic allocation, global lifetime
      [stack area] -function frame, local lifetime
    - <Operating System>
      - <memory system>
      memory layout
      [physical address]: absolute address
      [logical/virtual address]: relative address
      - <File system>
      -drive layout
      [block]
      -smallest file size unit
      [file]
      -file extension
      [directory (folder)]
      -file address table
      - <drive_partition method>
      [FAT(File Allocation Table) (file system)]
      store block addresses in linked list structure
      [ntfs (file system)]
      store block data and addresses in tree structure
      - <I/O system>
      - <drive>
      software-hardware interface
      - <hardware>
      - <CPU>
      - <CPU instruction cycle>
      [instruction fetch]
      [opcode decoding]
      [operand fetch]
      [instruction executed]
      [result store]
      - <ALU>
      -calculation
      - <control unit>
      -decoding
      - <register>
      [Program Counter] next opcode address
      [Instruction Register] -opcode
      [Memory Buffer Register] -value operand
      [Memory Address Register] address operand
      [general purpose register] -operand
      [flag register] -additional result state
      [stack pointer] -current stack address
      [base register] -base address
      - <CPU access>
      [register] 0~1 CPU cycle
      [cache] 1~10 cycle
      [memory] dozens ns
      [drive] hundreds ns
      - <memory>
      - <Random Access Memory>
      -빠른 접근을 위한
      -읽기 쓰기 모두
      - <Read Only Memory>
      -정해진 일 비드웨어 기본 동작을 위한
      -일반 읽기 쓰기 모두
      - <hardware>
      - <connection>
      hardware-hardware interface
      - <drive>
      -사용할 데이터 저장을 위한 장치
      - <Network Interface Card>
      [LAN card]
      - 유선 NIC
      [WIFI card]
      - 무선 NIC
- <network>
  - <protocol>
    통신 규약
    [OSI 7계층]
    -표준화된 네트워크 처리 구조
    [POI(Protocol Data Unit)]
    -처리할 정보의 단위
    - <application layer protocol>
    [HTTP(HyperText Transfer Protocol)]
    -인터넷을 위하여이 없는 하이퍼 프로토콜
    [URI(Uniform Resource Identifier) =URLS]
    [DNS(Domain Name System)]
    -ip 자장 서비스
    [DHCP]
    -IP주소 할당 서버에
    -ip 자장 제공 할당 서비스
    [SSH(Secure Shell)]
    -데이터 통신 서버
    -(암호화된) 원격 접속 서비스
    [SMTP]
    -e-mail 서비스
    [FTP]
    -file 전송 서비스
    - <Transport layer protocol>
    [TCP(Transmission Control Protocol)]
    -connectivity, reliability
    [P2P, multiplayer ...]
    [UDP(User Datagram Protocol)]
    -non-connectivity, non-reliability
    [broadcast, streaming service ...]
    - <network layer protocol>
    [IP(Internet Protocol)]
    -packet transmission
    [ICMP, ARP, IGMP, ...]
    [RIP (1 OSPF ... BGP ...]
    -route selection algorithm
    - <ethernet>
    [IEEE 802.3 protocol]
    -define encapsulation
    [CSMA/CD protocol]
    -separate collision domain in wire network
    -*wireless network
    -IEEE 802.11
    -CSMA/CA
    - <node>
    [DTE(Data Terminal Equipment)]
    -device that is the source or sink
    -client -requester
    -server -responder
    [DCE(Data Circuit-terminating Equipment)]
    -converting signal
    -intermediary device
    - <4.application layer>
    -L7(application layer)
    -L6(presentation layer)
    -L5(session layer)
    -POU: message (byte unit)
    - <socket>
    -network API
    - <4.transport layer>
    -POU: segment(TCP), datagram(UDP)
    [port number]
    -process identifier
    [connectivity]
    -check for communication with 3-way handshake
    [reliability]
    -ensures no data is missing
    - <3.network layer>
    -POU: packet
    [IP address]
    -Link identifier
    [intermediary device]
    -router -separate 2 connect [broadcast domain]s
    - <2.data link layer>
    -POU: frame
    [MAC address]
    -node identifier
    [line access]
    -P2P (1:1)
    -multipoint (1:n)
    [intermediary device]
    -switch: separate 2 connect [collision domain]s
    -VLAN: separate broadcast domain through a switch]
    - <bits>
    -node-node interface
    - <1.physical layer>
    -POU: bits
    [physical transmission line]
    -wire(metal cable)
    -wireless
    [line connection]
    [network topology]
    -mesh: 모든 노드가 서로를 연결
    -star: 중앙 노드가 모든 노드를 연결
    -bus: 하나 노드가 하나의 회선에 연결
    -tree: 하위 star가 bus에 연결
    -ring: 인접한 노드끼리만 서로를 연결
    [intermediary device]
    -repeater: signal amplification
    -hub: connection amplification
    -modem: digital || analog signal modulation
  - <techniques>
    - <cloud architecture>
    [on-premise]
    -non cloud
    [Infrastructure as a Service]
    = virtualization server, storage, network
    [Platform as a Service]
    -runtime, OS, middleware(WAS) + IaaS
    [Software as a Service]
    -business logic, resource + PaaS
    - <cloud systems>
    -multipoint 구조의 클럽 형태의 분산 처리 시스템
    - <distributed processing system>
    -네트워크 내 노드들이 분산 처리한 분산 작업을 최고 이용 공유하는 시스템
    - <blockchain>
    -P2P 구조의 분산 데이터베이스 관점 시스템
    -트랜잭션(거래) 정보 시 블록체인 프로토콜은 연결된 모든 노드에 배당 트랜잭션이 프로토콜시스템의 블록 생성 조건일정 시킨, 트랜잭션 수, 블록 생성 시간 간격 등 블록 생성 조건을 만족하면 블록을 생성하고, 새 블록을 브로드캐스트하여 전체 노드로부터 동의를 얻고 정상 그 이전 블록의 연결 리스트 형태로 저장하여 블록체인을 이룬다.
    이러 과정은 모든 노드가 새 블록 생성에 참여하고, 가장 먼저 유효한 블록을 생성한 노드에게 인센티브를 인센티브로 주는데 이를 코인이라고 불러요.
    [이더리움] 가장 많은 전자화폐를 가진 일부 노드를 대상으로 무작위 추첨하여 선정된 노드가 새 블록 생성을 담당한다.
    - <HTTP message>
    - <HTTP start-line>
    - <request start-line>
    [HTTP-method] URL HTTP-version
    - <HTTP method>
    [GET] (URLIE) 리소스 조회
    [POST]: (body)의 새로운 리소스 생성
    [PUT]: (body)의 리소스 전체 교체, 없으면 생성
    [PATCH]: (body)의 리소스 부분 교체
    [DELETE] (URLIE) 리소스 삭제
    - <URI(Uniform Resource Identifier)>
    [resource] server data
    [URL(Uniform Resource Locator)]
    -protocol://host:port/path?query#fragment
    -path/variable: /kim/10 정해진 주소로 이동하여 정보를 찾기(조회 등)
    -QueryParam: body가 아닌 URL에 데이터를 넣어 전송
    [URN(Uniform Resource Name)]
    -이름이기 없음
    - <response start-line>
    -상태에 대한 정보 및 결과
    HTTP-version State-code Reason-phrase
    - <state code>
    [1xx(Informational)]
    -OK(200) 요청이 정상적 처리 성공
    -Created(201) 새 리소스를 생성하는 성공 상태
    [3xx(redirection)]: 요청한 내용이 다른 URI로 리다이렉션
    -Moved Permanently(301): 요청 URI가 다른 로케이션 영구 redirection
    -Found(302): 요청 URI가 임시로 다른 로케이션 redirection
    -Not Modified: 요청한 리소스가 변경되지 않았기 때문에 redirection
    [4xx(client error)]
    -Unauthorized(401): 인증 필요(로그인 안함)
    -Forbidden(403): 인가 불가(접근 권한 없음)
    -Not Found(404): 요청 리소스가 존재하지 않음
    [5xx(server error)]
    -Internal Server Error(500): 서버 내부 문제
    - <HTTP/Hot Rubice Cat>
    [HTTP/1.1] 요청 한 번 보내고 응답 받아 끝나야만 다 해야지를 새로고침 해야 하는데 요청이 끝나지 않고계 연결하는 방식
    - <response header>
    [Server]: origin 서버의 소프트웨어 정보
    [Date]: 응답이 이뤄
    [Location]: URI to redirect
    [Content-Range]: 일부 전송한 범위 표시
    [Transfer-Encoding]: body 분할 전송 방식 지정
    [Content-Length:이용량]
    [WWW-Authenticate]: 401 응답 시서버가 요구하는 인증 방식 정보
    [Set-Cookie]
    server가 client에 데이터 기억는 고유한 값 전송
    [client] 받았던 cookie를 cookie storage에 저장
    -name=value; Expires=Date; Path=/path; ...
    -domain: 쿠키를 제공하는 서버 도메인 주소
    -path: domain의 상위 경로
    -expires/max-age: 쿠키 만료일(expires)/남은 시간(초)까지 유효시 소멸)
    -Secure: HTTPS 연결에서만 쿠키 전송
    -HttpOnly: 자바스크 쿠키 접근 차단
    [Last-Modified]: 최근 데이터에 대해서 데이터 수정일 표기
    [ETag]: 캐시 데이터에 대해 변경사항 확인용 해시 표기
    - <request header>
    [Host]: 요청하는 서버의 도메인 주소
    [Referer] 요청하는 URL 직전에 있던 페이지 URL
    [User-Agent]: Client가 사용 중인 OS, rendering engine, web-browser 정보
    [Accept]:Client가 선호하는 Content-Type
    [Accept-Encoding]:Client가 선호하는 Content-Encoding
    [Accept-Language]:Client가 선호하는 Content-Language
    [Range]: 범위 전송 구간 지정
    [Authorization]: 로그인 인증 정보
    [Cookie]: 브라우저 cookie storage에 보관되어 cookie를 서버에 전달
    [If-Modified-Since]: 서버에게 요청한 리소스가 변경되었는지 날짜가 수정된 경우에만 body를 전송 (아니면 304 응답)
    [If-None-Match]: etag를 통해 리소스가 변경되었는지 확인하는 수정된 경우에만 body를 전송 (아니면 304 응답)
    - <representation header>
    [Content-Type]: body 데이터 형식
    [Content-Encoding]:body 압축 전송 방식
    [Content-Language]: body 자연 언어 표시
    [Content-Length]: body 바이트 길이
    - <request body type>
    [x-www-form-urlencoded]
    -form form-data 전송 방식
    body로 전송되기전 query로 파싱
    [application/form]
    -API 통신용
    -가장 보편화된 데이터 포맷
    [application/ajax]
    -API 통신용
    -가장 보편화된 데이터 포맷
    - <response body type>
    [text/html]
    -html, css, js file
    [text/plain]
    -단순 텍스트 전송
    [application/ajax]
    -API 통신용
    -가장 보편화된 데이터 포맷
    [기타 등등]
    - <HTTP Header>
    -http message metadata
    (filed: value) 구조
    - <HTTP message>
    - <request>
    [request] 요청하는 서버의 요청한 주소
    [Referer] 요청하는 UR